

POLITECNICO
MILANO 1863

Actor Model – Akka Lab (2/2)

Luca Mottola

luca.mottola@polimi.it

https://mottola.faculty.polimi.it

Exercise 4

- Write a server actor that receives text messages and sends them back to the client actor
  - Upon receiving a **Sleep** message
    - It stores the subsequent messages
    - It stops sending messages back to the client
  - Upon receiving a **Wakeup** message
    - It sends the accumulated messages back to the client
      - In the same order in which they were received
    - It starts again to send new messages back to the client

Exercise 5

- Modify the solution to Exercise 3 so that the server actor fails every time a **PutMsg** is received with name "Fail!"
  - You may simply raise a generic Exception
- Create a supervisor actor that manages the fault
- Verify that using a **resume** strategy preserves the contact list, whereas a **restart** strategy effectively resets the contact list

Exercise 6

- Write a client actor that asks a server actor to uppercase one **TextMsg** at a time
  - The server actor replies after a short delay that emulates some longer processing
  - While processing, the server enters a “busy" behavior and buffers new requests arriving meanwhile
    - This means it must NOT block while processing
  - When processing a message completes, the server sends the reply to the client, becomes ready again, and processes buffered messages
  - For testing, create two clients issuing a series of commands in parallel
- You may use **getContext().system().scheduler().scheduleOnce()** in the solution, for example:

```
…scheduleOnce(delay, recipient, msg,
                        getContext().dispatcher(), sender);
```